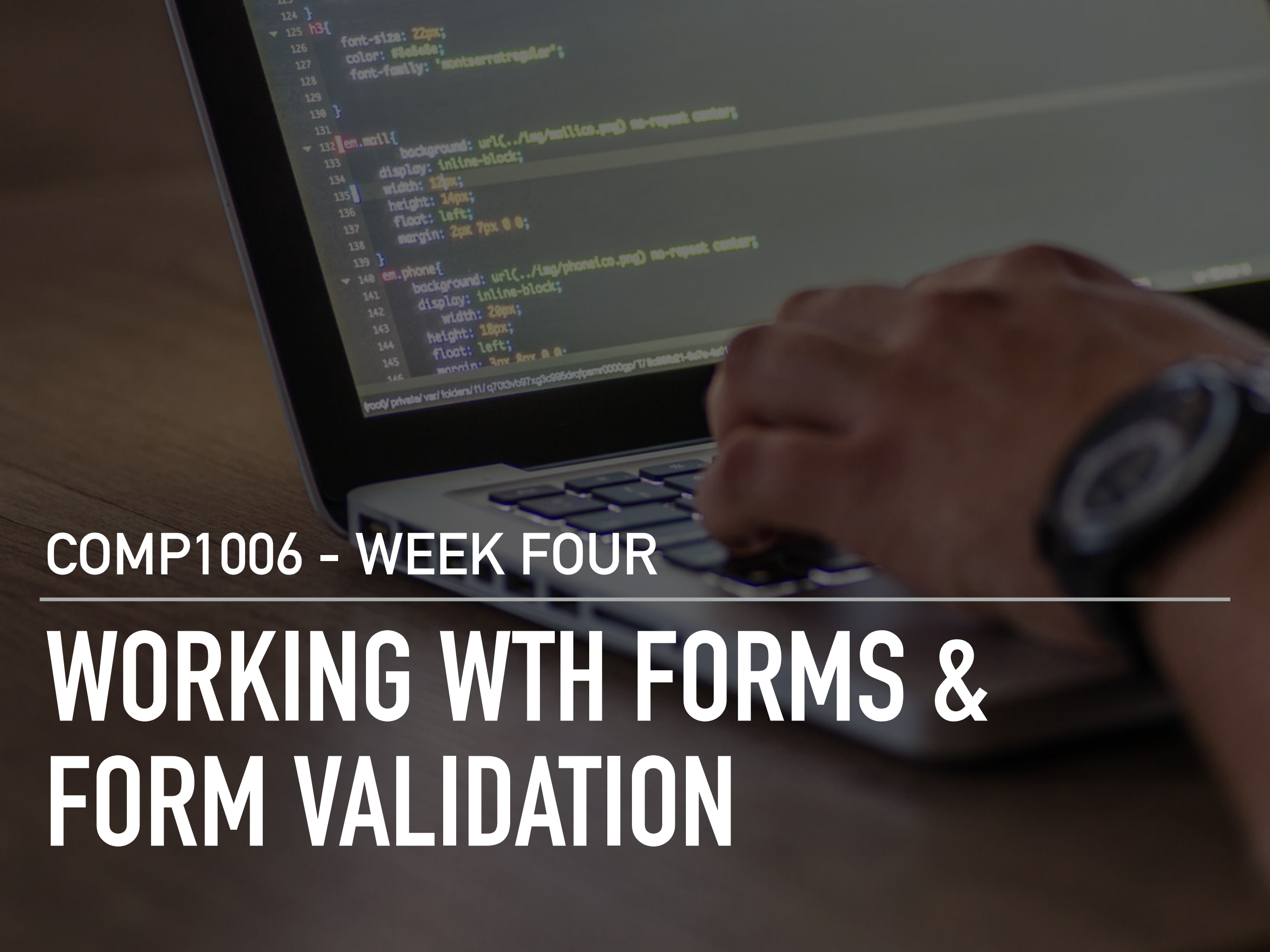


A close-up, slightly blurred photograph of a person's hand typing on a laptop keyboard. The laptop screen is visible in the upper left, displaying CSS code in a dark-themed editor. The code includes rules for 'h3', 'em.mail', and 'em.phone'. The background is a dark, textured surface.

COMP1006 - WEEK FOUR

---

# WORKING WITH FORMS & FORM VALIDATION

# TODAY'S SCHEDULE

1. Check In & Review
2. Review Lab Two
3. Working with Form Data
4. Form Validation
5. A Little Bit About Bootstrap
6. Lab Three
7. Summary, Task List, Next Week

```
JS get_users_payload.js  get_tweets_payload.py  JS get-user.js  JS get-tweet.js
Users > aspecker > Desktop > scripts > curl > JS get-user.js @accessToken
16 const accessTokenURL = new URL('https://api.twitter.com/oauth/access_token');
17 const authorizeURL = new URL('https://api.twitter.com/oauth/authorize');
18 const endpointURL = new URL('https://api.twitter.com/labs/1/users');
19
20 const params = {
21   usernames: 'AureliaSpecker',
22   format: 'detailed'
23 };
24
25 async function input(prompt) {
26   return new Promise(async (resolve, reject) => {
27     readline.question(prompt, (out) => {
28       readline.close();
29       resolve(out);
30     });
31   });
32 }
33
34 async function accessToken(oauth_token, oauth_token_secret, verifier) {
35   const oAuthConfig = {
36     consumer_key: ConsumerKey,
37     consumer_secret: ConsumerSecret,
38     token: oauth_token,
39     token_secret: oauth_token_secret,
40     verifier: verifier,
41   };
42
43   const req = await post({url: accessTokenURL, oauth: oAuthConfig});
44   if (req.body) {
45     return qs.parse(req.body);
46   } else {
47     throw new Error('Cannot get an OAuth request token');
48   }
49 }
50
51 async function requestToken() {
52   const oAuthConfig = {
53     callback: 'oob',
54     consumer_key: ConsumerKey,
55     consumer_secret: ConsumerSecret,
56   };
57
58   const req = await post({url: requestTokenURL, oauth: oAuthConfig});
59   if (req.body) {
60     return qs.parse(req.body);
61   } else {
62     throw new Error('Cannot get an OAuth request token');
63   }
64 }
```



# LEARNING OBJECTIVES

1. Capture user data from an HTML form
2. Validate user supplied data using client-side and server-side techniques
3. Utilize Bootstrap web framework for UI design



```
JS get_users_payload.js  get_tweets_payload.py  JS get-user.js  JS get-tweet.js
Users > aspecker > Desktop > scripts > URL > JS get-user.js @accessToken
16 const accessTokenURL = new URL('https://api.twitter.com/oauth/access_token');
17 const authorizeURL = new URL('https://api.twitter.com/oauth/authorize');
18 const endpointURL = new URL('https://api.twitter.com/labs/1/users');
19
20 const params = {
21   usernames: 'AureliaSpecker',
22   format: 'detailed'
23 };
24
25 async function input(prompt) {
26   return new Promise(async (resolve, reject) => {
27     readline.question(prompt, (out) => {
28       readline.close();
29       resolve(out);
30     });
31   });
32 }
33
34 async function accessToken(oauth_token, oauth_token_secret, verifier) {
35   const oAuthConfig = {
36     consumer_key: ConsumerKey,
37     consumer_secret: ConsumerSecret,
38     token: oauth_token,
39     token_secret: oauth_token_secret,
40     verifier: verifier,
41   };
42   const req = await post({url: accessTokenURL, oauth: oAuthConfig});
43   if (req.body) {
44     return qs.parse(req.body);
45   } else {
46     throw new Error('Cannot get an OAuth request token');
47   }
48 }
49
50
51 async function requestToken() {
52   const oAuthConfig = {
53     callback: 'oob',
54     consumer_key: ConsumerKey,
55     consumer_secret: ConsumerSecret,
56   };
57   const req = await post({url: requestTokenURL, oauth: oAuthConfig});
58   if (req.body) {
59     return qs.parse(req.body);
60   } else {
61     // ...
62   }
63 }
```

# WEEK THREE / LAB TWO REVIEW



# SYNTAX ERRORS

- ▶ prevent your application from **running**.
- ▶ they are the **easiest to find and fix**.
- ▶ PHP interpreter isn't able to parse the script, so it **displays an appropriate error message**.

# RUNTIME ERRORS

- ▶ does not violate the syntax rules, but they cause the PHP interpreter to display errors.
- ▶ may or may not stop execution of the script (fatal errors)
- ▶ the error message that's displayed usually can help you to find and fix the error.

# LOGIC ERRORS

- ▶ can come from many places, and they are often the most difficult to find and fix.
- ▶ Logic errors are statements that don't cause syntax or runtime errors
- ▶ produce the wrong result



# COMMON SYNTAX ERRORS

- ▶ Misspelling **keywords**.
- ▶ Forgetting an **opening or closing parenthesis, bracket, brace, or comment character**
- ▶ Forgetting to end a PHP statement with a **semicolon**.
- ▶ Forgetting an **opening or closing quotation mark**.
- ▶ Not using the **same opening and closing quotation mark**.

# VARIABLE NAMES

- ▶ Misspelling or incorrectly capitalizing a variable name.
- ▶ Using a keyword as a variable name.

# DEBUGGING TOOLS



- ▶ error messages
- ▶ var\_dump
- ▶ error\_logs

# WHY PROJECT STRUCTURE MATTERS

```
/project-root
  /includes
    header.php
    footer.php
  /assets
    css/
  index.php
```

- ▶ Separate concerns (logic vs presentation)
- ▶ Encourages reuse (DRY!)
- ▶ Prepares for larger apps
- ▶ **Hint: use include/require**



# LAB TWO REVIEW

```
124 }
125 h3{
126   font-size: 22px;
127   color: #8a8a8a;
128   font-family: 'montserratregular';
129 }
130 }
131
132 em.mail{
133   background: url(../img/mailico.png) no-repeat center;
134   display: inline-block;
135   width: 12px;
136   height: 14px;
137   float: left;
138   margin: 2px 7px 0 0;
139 }
140 em.phone{
141   background: url(../img/phoneico.png) no-repeat center;
142   display: inline-block;
143   width: 20px;
144   height: 18px;
145   float: left;
146   margin: 3px 8px 0 0;
147 }
```

# WORKING WITH FORM DATA

# WORKING WITH FORM DATA – WHAT COULD GO WRONG ?

Make the most of your professional life

Email

Password  
 Show

☒ Remember me

By clicking Agree & Join or Continue, you agree to the LinkedIn User Agreement, Privacy Policy, and Cookie Policy.

**Agree & Join**

or

## Re: Issues with the Order Form



**Lisa Dough** <lisa@bakeittillyoumakeit.com>

Today, 9:17 AM

Hi Team,

We're having some real headaches with the order form submissions! 🤔😓

- **Missing Information:** We're getting orders with no names, no phone numbers, or no addresses at all!
- **Weird Entries:** We're seeing "1234" for a name and "ASDF" as a phone number. Someone even put "Narnia" as an address! 🍷🌲❄️
- **Zero Quantities:** Some customers are submitting orders with *zero items* selected and nothing to bake! 💔😞

Plus! Could you have all order information sent to [bitumi@gmail.com](mailto:bitumi@gmail.com)?

It seems like we've been missing them!

To: [bitumi@gmail.com](mailto:bitumi@gmail.com)

Can you fix this ASAP? We can't keep guessing what people want, and it's causing chaos in the kitchen! 🧑🍳👩🍳

Thanks,

Lisa

**Lisa Dough**

[Bake It Till You Make It Bakery](#) 🏠

- ✓ An email was sent to [bitumi@gmail.com](mailto:bitumi@gmail.com)
- Subject: New bakery order submission
  - Attachments: Form data (txt file)

↩ Reply

➡ Forward

## WORKING WITH FORMS – ACTION & METHOD

1. Create a file called process.php. In that file, echo out a simple thank you message to the user. **What do you think should happen in process.php?**
2. Experiment changing the method in the form from post to get. **What do you notice?**



## WORKING WITH FORMS – ACTION & METHOD

1. Create a file called process.php. In that file, echo out a simple thank you message to the user. **What do you think should happen in process.php?**
2. Experiment changing the method in the form from post to get. **What do you notice?**

# THE <FORM> TAG: ACTION AND METHOD

```
<form action="process.php" method="post">
```

**Action:** where the form is sent

**Method:** how the information is sent

# METHOD = GET

```
<form action="process.php" method="post">
```

- ▶ Form data is sent in the URL
- ▶ Data is visible in the address bar
- ▶ Data can be bookmarked or shared
- ▶ Best for:
  - ▶ search forms
  - ▶ filters
  - ▶ non-sensitive data

# METHOD = POST

```
<form action="process.php" method="post">
```

- ▶ method="post"
- ▶ Form data is sent in the request body
- ▶ Data does not appear in the URL
- ▶ More secure than GET (but not encrypted)
- ▶ Best for:
  - ▶ orders
  - ▶ logins
  - ▶ forms with lots of data

**ADD VALIDATION ON THE CLIENT SIDE**

**CLIENT-SIDE VALIDATION WITH HTML5 ATTRIBUTES**



## HOW TO ACCESS FORM DATA?

**CHALLENGE :** TRY TO COMPLETE PROCESS.PHP. THE SCRIPT SHOULD GRAB FORM DATA AND MAIL ORDER TO THE CLIENT (LISA) [WITHOUT USING AI.]

WHAT DID YOU LEARN?

# WORKING WTH FORM DATA DEMO

# PHP SUPERGLOBALS

1. Built in PHP variables that hold associative arrays
2. `$_POST` & `$_GET`
3. Access form data using name attribute as the key value

# FORM VALIDATION

## Emails Working, But Still Getting Blank Forms



**Lisa Dough** <lisa@bakeittillyoumakeit.com>

Today, 9:24 AM

Hi Team,

Good news! The emails with our order forms are coming in just fine. 🥰

But...we're still getting some submissions where the forms are completely blank. 😓 Can somebody still send us a form without filling out any of the fields?

I'm starting to get a little concerned about the security of our form. Can we please look into this?

Thanks,  
Lisa

**Lisa Dough**

[Bake It Till You Make It Bakery](#) 🏠

↩ Reply   ↪ Forward

**WHAT'S WRONG WITH OUR FORM?**

# VALIDATING DATA



- ▶ Need to ensure that we are **receiving the right data**
- ▶ Client side validation is for **UX**, server-side validation is for **security**



# SANITIZING DATA

- ▶ Data supplied by the user is insecure
- ▶ Removing **illegal characters** from the data
- ▶ PHP provides a native filter extension that you can use to sanitize data and validate external input (**filter\_var, filter\_input**)

# ADD VALIDATION ON THE SERVER SIDE

# SERVER SIDE VALIDATION DEMO

```
124 }
125 h3{
126   font-size: 22px;
127   color: #8a8a8a;
128   font-family: 'montserratregular';
129 }
130 }
131
132 em.mail{
133   background: url(../img/mailico.png) no-repeat center;
134   display: inline-block;
135   width: 12px;
136   height: 14px;
137   float: left;
138   margin: 2px 7px 0 0;
139 }
140 em.phone{
141   background: url(../img/phoneico.png) no-repeat center;
142   display: inline-block;
143   width: 20px;
144   height: 18px;
145   float: left;
146   margin: 3px 8px 0 0;
147 }
```

# A LITTLE MORE BOOTSTRAP

Thanks for Securing the Form!



**Lisa Dough** <lisa@bakeittillyoumakeit.com>

Today, 10:17 AM

Hi Team,

I just wanted to thank you for securing our online order form and making sure everything works as it should. It's looking great! 💖

I really appreciate you making it safer and preventing those blank orders from coming through. 🛡️ Now I can relax knowing our form is secure and we're getting complete, legit orders. 🍰

It really makes a big difference for our bakery. Thank you so much for all your hard work! 😊

One more thing: when you get a chance, could we make our form look a little nicer? We don't have a lot of dough to spend, but even a small makeover would be sweet. 🍰 💵

Thanks again,

**Lisa Dough**

[Bake It Till You Make It Bakery](#) 🏠

↩ Reply

➡ Forward

HOW COULD YOU **IMPROVE** THE LOOK OF THIS  
FORM **WITHOUT IT COSTING THE CLIENT A LOT**  
**OF \$\$\$**

# A LIL' BIT ABOUT BOOTSTRAP



- ▶ a **framework** for building responsive, mobile-first websites & applications
- ▶ Excellent for **rapid development**
- ▶ **Easy** to use & integrate
- ▶ Some **drawbacks**



# BOOTSTRAP DEMO

```
124 }
125 h3{
126   font-size: 22px;
127   color: #8a8a8a;
128   font-family: 'montserratregular';
129 }
130 }
131
132 em.mail{
133   background: url(../img/mailico.png) no-repeat center;
134   display: inline-block;
135   width: 12px;
136   height: 14px;
137   float: left;
138   margin: 2px 7px 0 0;
139 }
140 em.phone{
141   background: url(../img/phoneico.png) no-repeat center;
142   display: inline-block;
143   width: 20px;
144   height: 18px;
145   float: left;
146   margin: 3px 8px 0 0;
147 }
```

# LAB THREE

# SUMMARY, TO DO & NEXT WEEK



# SUMMARY

1. HTTP requests (get & post) and working with Superglobals
2. Form Validation - Client Side & Server Side
3. A Little Bit Of Bootstrap

```
JS get_users_payload.js  get_tweets_payload.py  JS get-user.js  JS get-tweet.js
Users > aspecker > Desktop > scripts>URL > JS get-user.js > @accessToken
16 const accessTokenURL = new URL('https://api.twitter.com/oauth/access_token');
17 const authorizeURL = new URL('https://api.twitter.com/oauth/authorize');
18 const endpointURL = new URL('https://api.twitter.com/labs/1/users');
19
20 const params = {
21   usernames: 'AureliaSpecker',
22   format: 'detailed'
23 };
24
25 async function input(prompt) {
26   return new Promise(async (resolve, reject) => {
27     readline.question(prompt, (out) => {
28       readline.close();
29       resolve(out);
30     });
31   });
32 }
33
34 async function accessToken(oauth_token, oauth_token_secret, verifier) {
35   const oAuthConfig = {
36     consumer_key: ConsumerKey,
37     consumer_secret: ConsumerSecret,
38     token: oauth_token,
39     token_secret: oauth_token_secret,
40     verifier: verifier,
41   };
42
43   const req = await post({url: accessTokenURL, oauth: oAuthConfig});
44   if (req.body) {
45     return qs.parse(req.body);
46   } else {
47     throw new Error('Cannot get an OAuth request token');
48   }
49 }
50
51 async function requestToken() {
52   const oAuthConfig = {
53     callback: 'oob',
54     consumer_key: ConsumerKey,
55     consumer_secret: ConsumerSecret,
56   };
57
58   const req = await post({url: requestTokenURL, oauth: oAuthConfig});
59   if (req.body) {
60     return qs.parse(req.body);
61   } else {
```

**TO DO THIS WEEK :**

**LAB THREE (DUE FRIDAY, JAN 30TH @ 11:59PM)**

**NEXT WEEK :**

**CREATE & READ**